

Gradient Descent

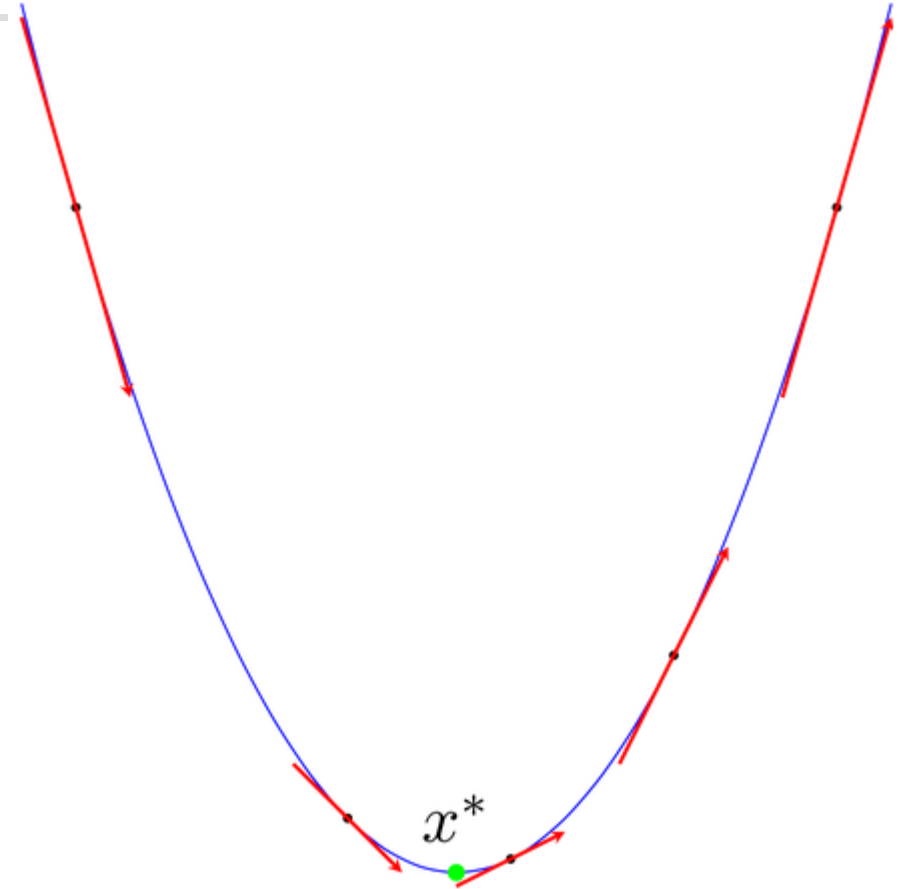
TS. Nguyễn Thị Kim Ngân

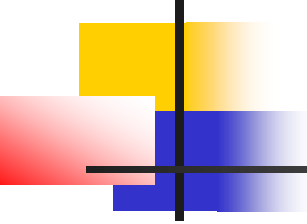
Introduction

$$f(x) = \frac{1}{2}(x - 1)^2 - 2$$

x	$-\infty$	1	$+\infty$
$f'(x)$	-	0	+
$f(x)$	$+\infty$		$+\infty$

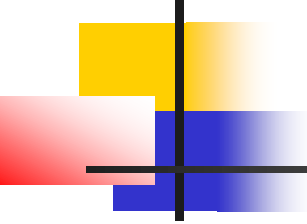
-2





Introduction

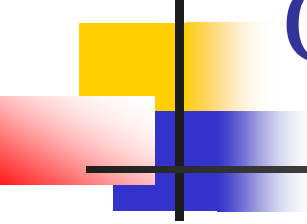
- Local minimum: a point where the derivative of the function equals zero
- Global minimum: a point where the function reaches its smallest possible value



Introduction

Consider a single-variable function $f(x)$ that is differentiable everywhere:

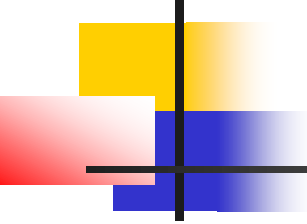
- A local minimum x^* of the function is a point where $f'(x^*)=0$
- In the neighborhood of x^*
 - The derivative at points to the left of x^* is negative.
 - The derivative at points to the right of x^* is positive.
- The tangent line to the graph of the function at x_t has a slope equal to the derivative of the function at x_t



Gradient Descent

In machine learning, we often need to find the minimum value of a function

- The minimum points are the solutions of the derivative equation equal to 0.
- If all (finite) minimum points can be found, we only need to substitute each local minimum into the function and determine which one gives the smallest value

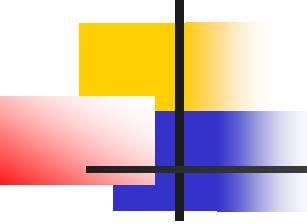


Gradient Descent

Solving a zero derivative equation is often very complicated or may yield infinitely many solutions. Because:

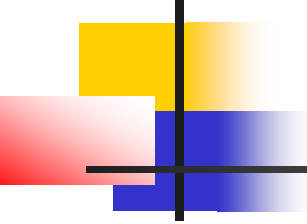
- Complexity of the form of the derivative
- High-dimensional data points
- Too many data points

=> **Therefore, finding the global minimum of the function may be impossible**



Gradient Descent

- People often try to find local minimum points, and to some extent, consider them as the desired solution of the problem
- In practice, in many machine learning problems, local minimum solutions often give good results, especially in neural networks



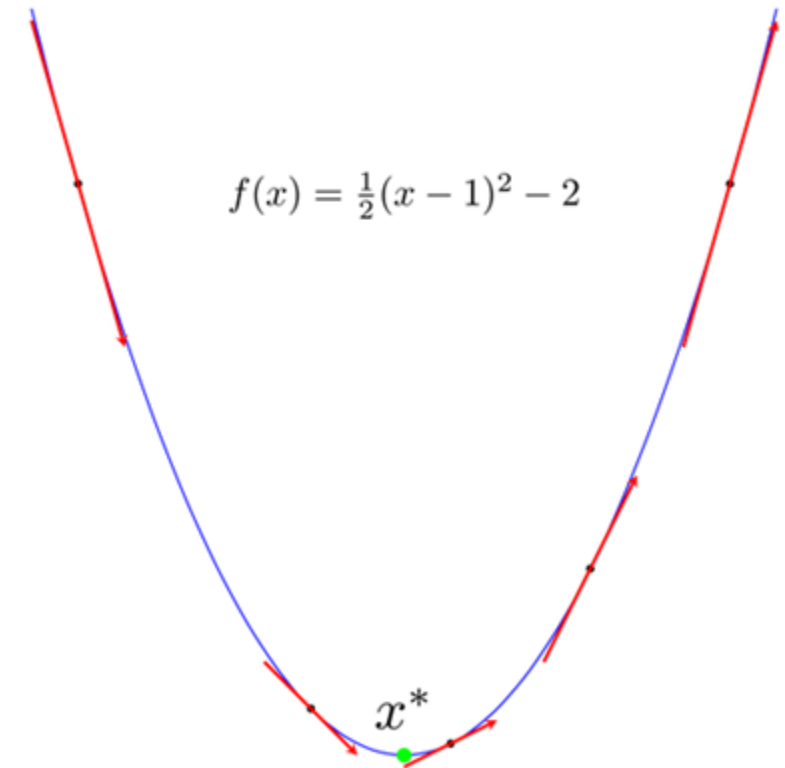
Gradient Descent

Gradient Descent is one of the most commonly used methods to find the minimum point of an optimization problem:

- Starting from a point considered close to the solution of the problem
- Using an iterative operation to gradually approach the desired point (with derivative close to 0)

Gradient Descent cho hàm 1 biến

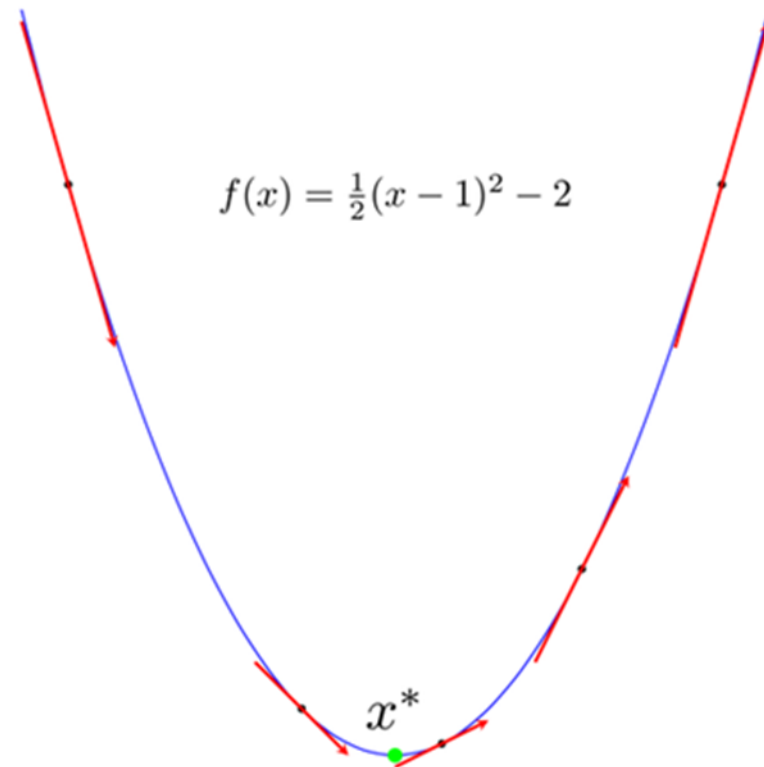
Suppose x_t is the point obtained after the t^{th} iteration. We need to find an algorithm that brings x_t as close as possible to x^*



Gradient Descent for single-variable functions

- If $f'(x_t) > 0$, then x_t lies to the right of x^* , and vice versa
- The farther x_t is to the right of x^* , the larger $f'(x_t)$ is than 0 (and vice versa)
- To make the next point x_{t+1} closer to x^* , we need to move x_t towards the opposite sign of the derivative:

$$x_{t+1} = x_t - f'(x_t)$$

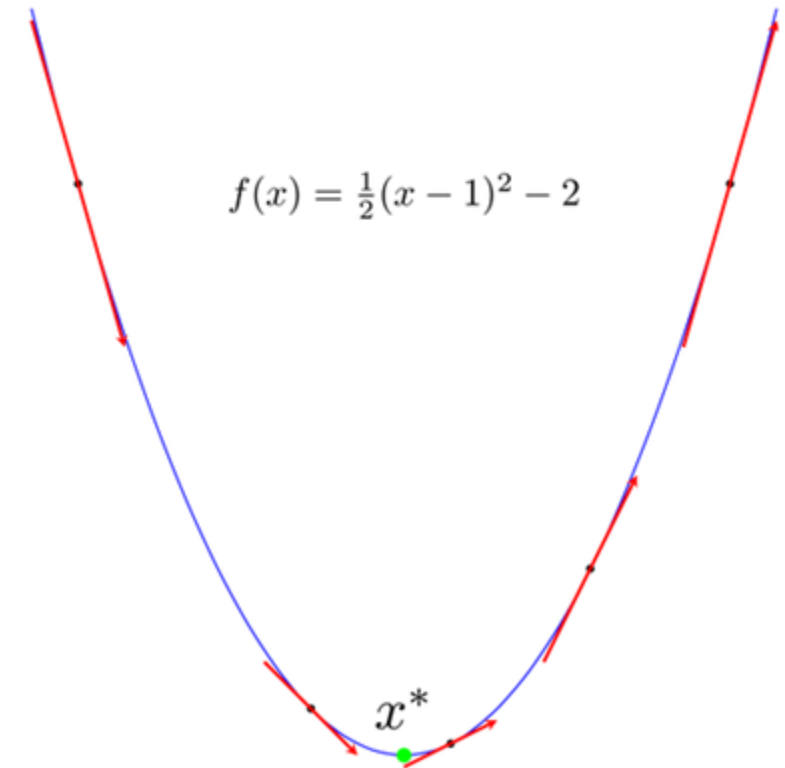


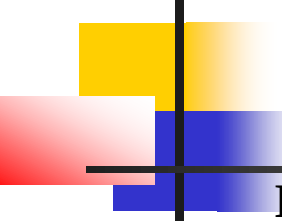
Gradient Descent for single-variable functions

General formula:

$$x_{t+1} = x_t - \eta f'(x_t)$$

Where η is a positive number called the **learning rate**





Finding an Approximate Solution to an Optimization Problem Using Gradient Descent

Input: Training dataset D_{train} , loss function $J(w)$, learning rate η , convergence threshold ϵ , and maximum number of iterations $Iteration_max$

Output: Optimal parameter set w^*

Step 1: Randomly initialize w

Step 2: Set $iteration=0$

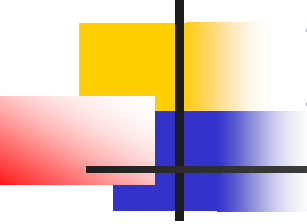
Step 3: Repeat until the gradient of the loss function is approximately zero or the maximum number of iterations is reached.

while $|\nabla J(w)| > \epsilon$ and $iteration < Iteration_max$ **do**

- Compute the loss function $J(w)$
- Compute the gradient: $g = \nabla J(w)$
- Update the parameters: $w \leftarrow w - \eta g$
- Increment the counter: $iteration = iteration + 1$

end while

Step 4: Return $w^* = w$

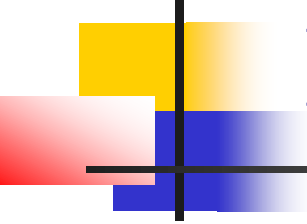


Example

Let $f(x)=x^2+5\sin(x)$

- $f'(x)=2x+5\cos(x)$
- Starting from a point x_0 , at the t^{th} iteration:

$$x_{t+1} = x_t - \eta(2x_t+5\cos(x_t))$$



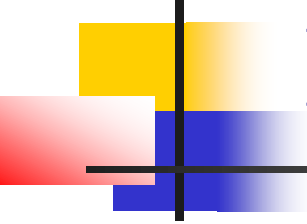
Example

Let $f(x)=x^2+5\sin(x)$

- $f'(x)=2x+5\cos(x)$
- Starting from a point x_0 , at the t^{th} iteration:

$$x_{t+1} = x_t - \eta(2x_t+5\cos(x_t))$$

- **cost** is used to compute the value of the function
- **grad** is used to compute the derivative
- **myGD1** is the main part that implements the GD algorithm described above. The inputs to this function are the learning rate and the starting point. The algorithm stops when the magnitude of the derivative is sufficiently small



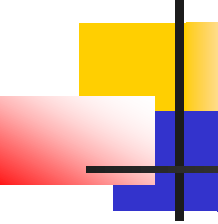
Example

Let $f(x)=x^2+5\sin(x)$

- $f'(x)=2x+5\cos(x)$
- Starting from a point x_0 , at the t^{th} iteration:

$$x_{t+1} = x_t - \eta(2x_t+5\cos(x_t))$$

```
def grad(x):  
    return 2*x+ 5*np.cos(x)  
  
def cost(x):  
    return x**2 + 5*np.sin(x)  
  
def myGD1(x0, eta):  
    x = [x0]  
    for it in range(100):  
        x_new = x[-1] - eta*grad(x[-1])  
        if abs(grad(x_new)) < 1e-3: # just a small number  
            break  
        x.append(x_new)  
    return (x, it)
```



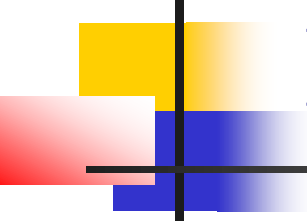
Example

Experiment with different initial points $x_0 = -5$, $x_0 = 5$, using the same learning rate $\eta=0.1$

```
(x1, it1) = myGD1(-5, .1)
(x2, it2) = myGD1(5, .1)
print('Solution x1 = %f, cost = %f, after %d iterations'%(x1[-1], cost(x1[-1]), it1))
print('Solution x2 = %f, cost = %f, after %d iterations'%(x2[-1], cost(x2[-1]), it2))
```

Results

```
Solution x1 = -1.110667, cost = -3.246394, after 11 iterations
Solution x2 = -1.110341, cost = -3.246394, after 29 iterations
```



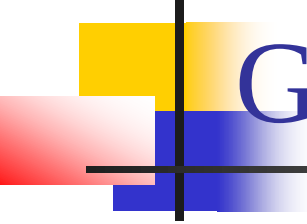
Exercises

Exercise 1: Find the minimum value of the function $f(x)=x^2-2$

Exercise 2: Find the minimum value of the function $g(x)=(1/3)x^3-x$

$$(a.x^n)'=n.(a.x^{n-1})$$

$$g'(x)=3.(1/3).x^2-1.x^0=x^2-1$$



Gradient Descent for a multivariable function

Seeking the global minimum of a function $f(w)$, where w is a vector

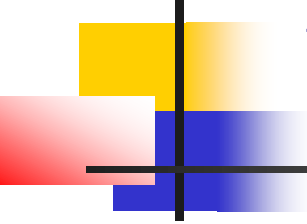
The derivative (gradient) of the function at any point w is denoted by $\nabla_w f(w)$

The Gradient Descent for a multivariable function $f(w)$:

- Start from an initial point w_0
- Iterate until the stopping criterion is met. The update rule at t -th iteration is:

$$w_t = w_{t-1} - \eta \nabla_w f(w)$$

$$\text{hoặc } w = w - \eta \nabla_w f(w)$$



Optimizing the loss function of Linear Regression using Gradient Descent

The loss function of Linear Regression:

$$f(w) = \frac{1}{2N} \|y - Xw\|_2^2$$

The derivative of the loss function is:

$$\nabla_w f(w) = \frac{1}{N} X^T (Xw - y)$$

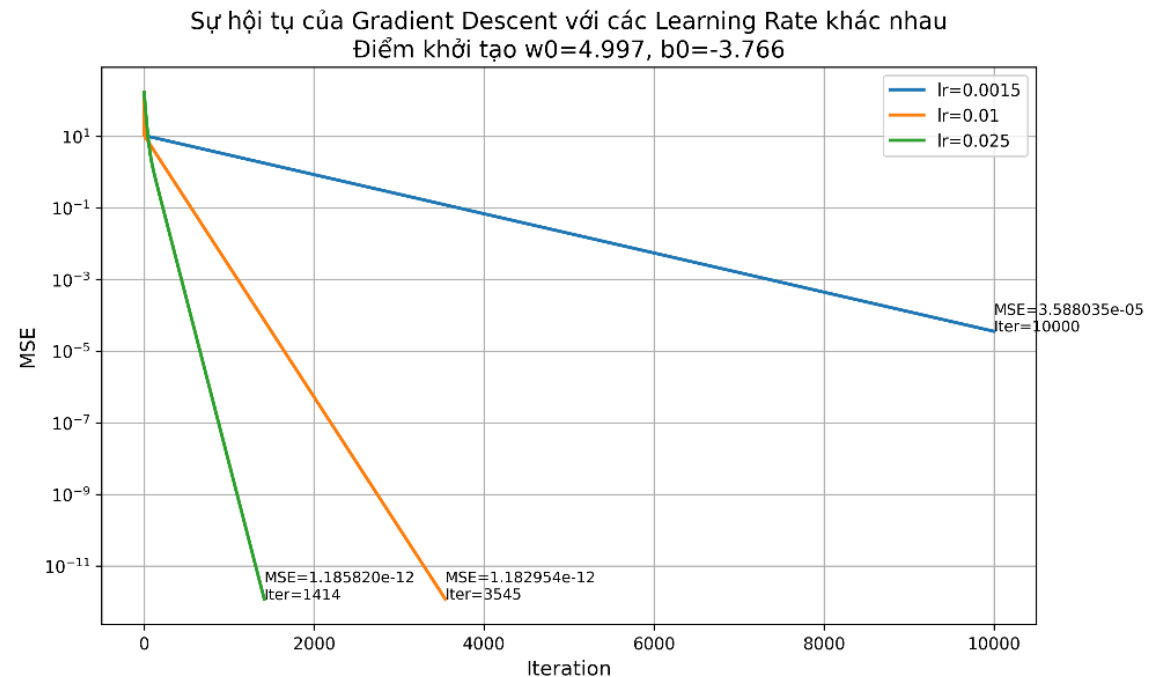
Optimizing the loss function using Gradient Descent

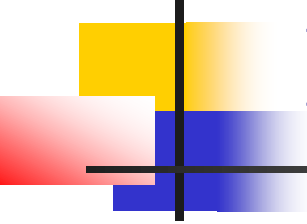
- Start from an initial point w_0
- Iterate until the stopping criterion is met. The update rule at t-th iteration is:

$$w = w - \eta \nabla_w f(w)$$

Learning rate

- **Small learning rate:** The algorithm converges slowly, the computational time increases. The number of iterations is large enough, the solution can get closer to the optimum. The number of iterations is insufficient, the algorithm may stop before reaching the optimum
- **Large learning rate:** The algorithm converges quickly, requiring fewer iterations. However, the parameters may move too far in each step, so the solution can remain far from the true optimum
- **Appropriate learning rate and number of iterations:** A suitable combination helps the algorithm reach a near-optimal solution with a reasonable computational time





Batch Gradient Descent

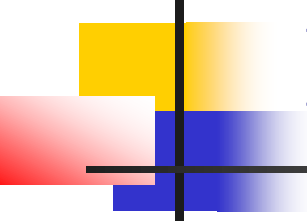
Each update step, the Gradient Descent algorithm uses the **entire training set** to compute the gradient

Given a data set $D=\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(N)}, y^{(N)})\}$, $\nabla J_j(w)$ is the derivative of the loss function with respect to (w) , evaluated at sample $x^{(j)}$, $j=1, 2, \dots, N$

At each iteration, the coefficient (w) of the loss function is updated as follows:

$$\nabla J(w) = \frac{1}{N} \sum_{j=1}^N \nabla J_j(w)$$

$$w \leftarrow w - \eta \nabla J(w)$$



Batch Gradient Descent

The loss function of the Linear Regression model at sample $x^{(j)}$

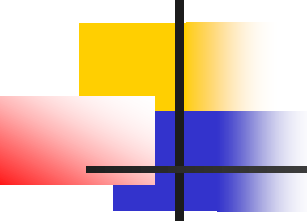
$$J_j(w) = \frac{1}{2} \left(y^{(j)} - (x^{(j)})^T w \right)^2$$

The derivative of the loss function with respect to (w) , evaluated at sample $x^{(j)}$

$$\nabla J_j(w) = x^{(j)} \left((x^{(j)})^T w - y^{(j)} \right)$$

At each iteration, the coefficient (w) of the loss function is updated as follows:

$$w \leftarrow w - \eta \frac{1}{N} \sum_{j=1}^N x^{(j)} \nabla J_j(w) = w - \eta \frac{1}{N} \sum_{j=1}^N x^{(j)} \left((x^{(j)})^T w - y^{(j)} \right)$$



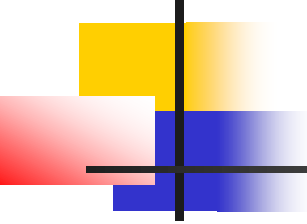
Stochastic Gradient Descent

Each update step, Stochastic Gradient Descent (SGD) **randomly selects one data sample** at each update of (w) to compute the gradient

Given a data set $D=\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(N)}, y^{(N)})\}$, $\nabla J_j(w)$ is the derivative of the loss function with respect to (w) , evaluated at sample $x^{(j)}$, $j=1, 2, \dots, N$

At each iteration, the coefficient (w) of the loss function is updated as follows:

$$w \leftarrow w - \eta \nabla J_j(w)$$



Stochastic Gradient Descent

The loss function of the Linear Regression model at sample $x^{(j)}$

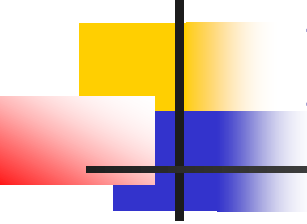
$$J_j(w) = \frac{1}{2} \left(y^{(j)} - (x^{(j)})^T w \right)^2$$

The derivative of the loss function with respect to (w) , evaluated at sample $x^{(j)}$

$$\nabla J_j(w) = x^{(j)} \left((x^{(j)})^T w - y^{(j)} \right)$$

At each iteration, the coefficient (w) of the loss function is updated as follows:

$$w \leftarrow w - \eta \nabla J_j(w) = w - \eta x^{(j)} \left((x^{(j)})^T w - y^{(j)} \right)$$



Mini-batch Gradient Descent

Mini-batch Gradient Descent (MBGD) **uses a small subset of the training data** to compute the gradient at each update of (w) . At the beginning of each epoch, Mini-batch Gradient Descent randomly shuffles the training data and then divides the entire dataset into mini-batches, each containing (k) data points.

Given a data set $D = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(N)}, y^{(N)})\}$, let $B \subseteq D$ be a mini-batch, and $\nabla J_{xy}(w)$ denote the derivative of the loss function with respect to (w) , evaluated at a sample $(x, y) \in B$

At each iteration, MBGD updates (w) as follows:

$$\nabla J_B(w) = \frac{1}{|B|} \sum_{(x,y) \in B} \nabla J_{xy}(w)$$

$$w \leftarrow w - \eta \nabla J_B(w)$$